

- This is a set of “simple” problems that someone who has *already*—properly!—done a course on Algorithms should have no difficulty solving correctly, in a single attempt.
 - That is: they may take some time to design a solution, but their *first implementation attempt* will be correct—without needing debugging or multiple submissions.
- These are also at the level that is used to filter *out* candidates in placement tests. Just being able to solve problems of this kind these won't get you a job, but not being able to solve these—correctly, and within the test's time limits—will most likely result in you *not* getting that job.
- My intention in sending these out now, is not to scare you. Instead, it is to give you a sense of where you stand, in comparison to the bare minimum that will be expected of you come placement time.
- Many of you *won't* be able to solve these correctly at this point of time. Don't worry: if you put in an honest effort to do the course well, you will be able to solve these—and vastly more difficult—problems correctly, by the time we are done with the course.
- You are not *required* to submit your solutions to these problems. You are free to take the help of the TAs (during the tutorials) to check your solutions/partial solutions/attempts.

1.

- (a) Write a Python3 function `FizzBuzz(x)` that takes a positive integer x as its argument, and returns the sum of all the positive integer multiples of 3 *or* 5 that are less than x .

E.g.: `FizzBuzz(10)` should return 23, and `FizzBuzz(1000)` should return 233168.

- (b) Test your Python code on a few hundred inputs.

2. The sequence of *Fibonacci numbers* is defined as follows:

$$f_1 := 1, f_2 := 2, f_{n+2} := f_{n+1} + f_n, \forall n \in \mathbb{N}$$

Thus the sequence goes:

1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

- (a) Write a Python3 function `EVENFIBSUM(x)` that takes a positive integer x as its argument, and returns the sum of all the *even* Fibonacci numbers which are smaller than x .
E.g.: `EVENFIBSUM(100)` should return 44, and `EVENFIBSUM(4000000)` should return 4613732.
- (b) Test your Python code on a few hundred inputs.
3. The *Collatz sequence* starting at a positive integer n is the sequence of positive integers $\langle a_0, a_1, a_2, \dots \rangle$ defined by:

$$a_0 = n, \quad a_{k+1} = \begin{cases} \frac{a_k}{2} & \text{if } a_k \text{ is even,} \\ 3a_k + 1 & \text{if } a_k \text{ is odd.} \end{cases}$$

The *length* of the Collatz sequence starting at n is the smallest integer $\ell \geq 0$ such that $a_\ell = 1$.

For example, the length of the Collatz sequence starting at $n = 13$ is 10, since the sequence goes:

$$13 \rightarrow 40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow \dots$$

- (a) Write a Python3 function `ARGMAXCOLLATZ(x)` that takes a positive integer x as its argument, and returns a positive integer y smaller than x for which the Collatz sequence starting at y has the maximum length among all positive integers less than x .
E.g.: `ARGMAXCOLLATZ(1000000)` should return 837799.
- (b) Test your Python code on a few hundred inputs.
4. A *permutation* is an ordered arrangement of objects. For example, 3124 is one possible permutation of the digits 1, 2, 3, 4. A *lexicographic order* of permutations of a set, is a list of all permutations of that set, ordered in “dictionary order”.

For instance, the lexicographic order of permutations of the digits 0, 1, 2 is:

$$012 \quad 021 \quad 102 \quad 120 \quad 201 \quad 210$$

- (a) Write a Python3 function `NTHPERMUTATION(n)` that takes a positive integer n as its argument, and returns the n th permutation of the digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, in lexicographic order.
E.g.: `NTHPERMUTATION(1000000)` should return 2783915460.
- (b) Test your Python code on a few hundred inputs.